

State

Each question is encoded as a dense semantic representation $s_i = f_{\text{emb}}(x_i) \in \mathbb{R}^d$, where f_{emb} is a frozen sentence encoder. In our implementation, we use a pretrained sentence-embedding model and represent each instance using either the question alone or the concatenation of passage and question, depending on the task format. The state therefore captures semantic information about the input instance while remaining independent of the target LLM parameters.

Action

The action space is discrete and consists of the six candidate emotions in $\mathcal{A}$. Given a state s_i , the policy outputs a categorical distribution $\pi_\theta(a_k | s_i)$ for $k = 1, \dots, K$, where $K = 6$. At inference time, the selected action is $a_i^* = \arg \max_{a_k \in \mathcal{A}} \pi_\theta(a_k | s_i)$. This action determines which emotion-conditioned prompt is applied before querying the frozen LLM.

Reward

To train the policy offline, we first evaluate the frozen LLM under all candidate emotions for each training instance and construct a grouped reward cache. For each question x_i , this yields a binary reward vector $\mathbf{r}_i = [r_i^{(1)}, \dots, r_i^{(K)}]$, where $r_i^{(k)} \in \{0, 1\}$ and $r_i^{(k)} = \mathbf{1}[\hat{y}_i^{(k)} = y_i]$. Here, y_i denotes the gold label and $\hat{y}_i^{(k)}$ denotes the backbone model prediction under emotion a_k . In the main setting used in our experiments, reward is defined directly by answer correctness.

Agent architecture and training algorithm

The policy is implemented as a lightweight two-layer MLP that maps the input embedding s_i to logits over the six emotions, $z_i = W_2 \sigma(W_1 s_i + b_1) + b_2$, followed by a softmax to obtain $\pi_\theta(a_k | s_i)$. Inspired by group-relative optimization methods such as GRPO, we construct supervision from the relative rewards of multiple candidate emotions for the same input, rather than treating each instance as having a single hard optimal action [45]. Instead of using a hard one-hot target, we convert the reward vector into a soft target distribution, following the intuition of soft-target supervision in knowledge distillation [46]. For each instance, we first center rewards by subtracting the mean reward across actions and then compute reward-induced weights,

$$w_i^{(k)} = \frac{\exp\left((r_i^{(k)} - \bar{r}_i)/\tau\right)}{\sum_{j=1}^K \exp\left((r_i^{(j)} - \bar{r}_i)/\tau\right)}, \quad (1)$$

where τ is a temperature hyperparameter. In our practical setting, we set $\tau = 1$ to provide a moderate degree of smoothing. Intuitively, $w_i^{(k)}$ assigns greater probability mass to emotions that yield higher task rewards for instance i , while still preserving graded preferences across actions. The policy is trained by minimizing the reward-weighted cross-entropy,

$$\mathcal{L} = - \sum_{i=1}^N \sum_{k=1}^K w_i^{(k)} \log \pi_{\theta}(a_k | s_i). \quad (2)$$

Here, $\mathcal{L}$ denotes the training loss, the outer summation aggregates over training instances, and the inner summation aggregates over candidate emotions for each instance. This objective encourages the policy distribution to align with the relative utility of different emotional framings, rather than collapsing the problem into hard classification with a single “best” label.

EmotionRL pipeline.

We instantiate EmotionRL on five benchmark datasets with official training splits: $\mathcal{D} \in \{\text{GSM8K}, \text{OPENBOOKQA}, \text{SOCIALQA}, \text{MEDQA}, \text{BOOLQ}\}$. We exclude BBH because it does not provide a standard supervised training split; incorporating it would require introducing a synthetic split, which would reduce comparability across benchmarks. For each training instance (x_i, y_i) , we enumerate all candidate emotions $a_k \in \mathcal{A}$, where $|\mathcal{A}| = K = 6$, construct the corresponding emotion-conditioned prompts, and query the frozen backbone LLM under each condition. This yields a grouped reward vector $\mathbf{r}_i = [r_i^{(1)}, \dots, r_i^{(K)}]$, where $r_i^{(k)} = \mathbf{1}[\hat{y}_i^{(k)} = y_i]$ indicates whether the backbone model prediction under emotion a_k is correct. Aggregating these reward vectors over the official training split produces an offline reward dataset $\{(s_i, \mathbf{r}_i)\}_{i=1}^N$, where $s_i = f_{\text{emb}}(x_i)$ is the frozen embedding of input x_i .

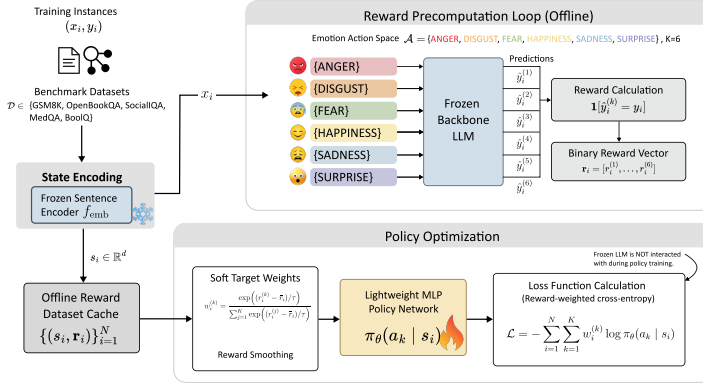
The policy network $\pi_{\theta}(a_k | s_i)$ is then trained on the official training split by minimizing the reward-weighted objective in Eq. (2), where the soft target weights $\{w_i^{(k)}\}_{k=1}^K$ are computed from $\mathbf{r}_i$ via Eq. (1). Thus, training is performed fully offline: the policy never interacts with the backbone model during optimization, but instead learns from the precomputed reward table over candidate emotions.

At test time, for each unseen input x , we first compute its embedding $s = f_{\text{emb}}(x)$. Then use the trained policy to select the emotion

$$a^* = \arg \max_{a_k \in \mathcal{A}} \pi_{\theta}(a_k | s). \quad (3)$$

The selected emotion a^* is used to construct the final emotion-conditioned prompt, which is then submitted once to the frozen backbone LLM to obtain the prediction $\hat{y}$. All reported EmotionRL results therefore reflect an online deployment setting in which the policy performs instance-level emotion selection before the backbone model generates its final answer.

(A) Offline Training Phase



(B) Online Inference Phase

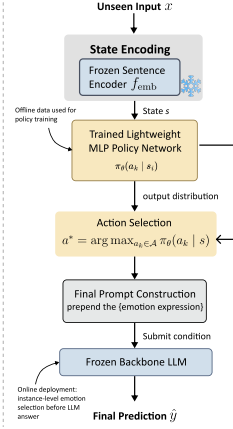


Fig. 2 EmotionRL pipeline for adaptive emotion selection. **a, Offline training.** For each training instance (x_i, y_i) from the benchmark datasets, EmotionRL first computes a frozen semantic state $s_i = f_{\text{emb}}(x_i)$. It then enumerates all candidate emotions $a_k \in \mathcal{A}$, constructs the corresponding emotion-conditioned prompts, and queries the frozen backbone LLM under each condition. This produces a grouped reward vector $\mathbf{r}_i = [r_i^{(1)}, \dots, r_i^{(K)}]$, where $r_i^{(k)} = \mathbf{1}[\hat{y}_i^{(k)} = y_i]$. The reward vector is converted into soft supervision $w_i^{(k)}$ and cached as an offline reward dataset $\{(s_i, \mathbf{r}_i)\}_{i=1}^N$. A policy network is then trained to predict $\pi_\theta(a_k | s_i)$ by minimizing the reward-weighted cross-entropy objective in Eq. (2). **b, Online inference.** For an unseen input x , EmotionRL computes $s = f_{\text{emb}}(x)$, applies the trained policy $\pi_\theta(a_k | s)$, and selects $a^* = \arg \max_{a_k \in \mathcal{A}} \pi_\theta(a_k | s)$. The selected emotion a^* is then used to construct a single emotion-conditioned prompt, which is submitted once to the frozen backbone LLM to obtain the final prediction $\hat{y}$.

3 Results

We organize the empirical findings around four questions: whether static emotional prefixes change task accuracy, whether stronger affective wording amplifies the effect, whether the source of the emotional text matters, and whether adaptive emotion selection can exploit any useful signal.

3.1 Static emotional prefixes have limited average effect across tasks

Figure 3 shows the main empirical result of the paper: across a diverse benchmark suite, prepending emotional text rarely produces large changes in task accuracy. The dominant pattern is stability. In most task-model pairs, emotional framing neither substantially improves nor substantially degrades performance relative to the neutral baseline. Static emotional prompting therefore does not behave like a reliable general-purpose method for improving model performance.

The degree of sensitivity is nevertheless task-dependent. GSM8K and MedQA-US remain especially close to baseline across emotions, suggesting that short affective